HelloWorld.r

```r
make_sensor_dataframe <- function(sensor_id, voltage, status) {

  vectors <- list(
    sensor_id = sensor_id,
    voltage   = voltage,
    status    = status
  )

  lengths <- sapply(vectors, length)

  expected_length <- max(lengths)

  cat("===== SENSOR VECTOR DIAGNOSTIC REPORT =====\n\n")

  cat("Vector lengths:\n")
  print(lengths)

  cat("\nExpected number of observations:",
      expected_length, "\n\n")

  shorter_vectors <- names(lengths[lengths < expected_length])

  if (length(shorter_vectors) == 0) {

    cat("No vector-length discrepancy detected.\n")

  } else {

    cat("Length discrepancies detected:\n\n")

    for (name in shorter_vectors) {

      current_length <- lengths[name]
      missing_count <- expected_length - current_length

      missing_positions <-
        seq(current_length + 1, expected_length)

```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
sensor_id   voltage    status
        4         2         1

Expected number of observations: 4

Length discrepancies detected:

Vector: voltage
Current length: 2
Expected length: 4
Number of missing observations: 2
Missing index position(s): 3, 4

Vector: status
Current length: 1
Expected length: 4
Number of missing observations: 3
Missing index position(s): 2, 3, 4

===== AFTER RECTIFICATION =====

Final vector lengths:
sensor_id   voltage    status
        4         4         4
===== FINAL DATA FRAME =====

  sensor_id voltage status
1        A1     4.8 Active
2        B2     5.0   <NA>
3        C3      NA   <NA>
4        D4      NA   <NA>
```

```r
clean_temperature <- function(df) {

  cat("===== TEMPERATURE DATA DIAGNOSTIC REPORT =====\n\n")

  cat("Original column class:",
      class(df$Temperature), "\n\n")

  numeric_values <- suppressWarnings(
    as.numeric(df$Temperature)
  )

  non_numeric <- is.na(numeric_values) &
                 !is.na(df$Temperature)

  error_indices <- which(non_numeric)

  if (length(error_indices) == 0) {

    cat("No non-numeric temperature tokens detected.\n")

  } else {

    cat("Non-numeric token(s) detected:\n\n")

    token_frequency <- table(df$Temperature[error_indices])

    for (token in names(token_frequency)) {

      token_rows <- which(
        df$Temperature == token
      )

      cat("Token:", token, "\n")
      cat("Frequency:", token_frequency[token], "\n")
      cat("Row index position(s):",
          paste(token_rows, collapse = ", "),
          "\n\n")
    }
```

STDIN

Input for the program (Optional)

Output:

```
===== TEMPERATURE DATA DIAGNOSTIC REPORT =====

Original column class: character

Non-numeric token(s) detected:

Token: ERR_99
Frequency: 2
Row index position(s): 3, 6

===== AFTER RECTIFICATION =====

Final column class: numeric
Number of NA values: 2

Final Temperature values:
[1] 24.5 25.1   NA 26.3 27.0   NA 28.2 29.1
===== FINAL DATA FRAME =====

  Sensor_ID Temperature Status
1       S01        24.5     OK
2       S02        25.1     OK
3       S03          NA  ERROR
4       S04        26.3     OK
5       S05        27.0     OK
6       S06          NA  ERROR
7       S07        28.2     OK
8       S08        29.1     OK

===== FINAL MEAN TEMPERATURE =====
```

```r
reshape_strain_data <- function(df, time_col, sensors) {

  coordinate_names <- c("X", "Y")

  result <- data.frame(
    Time = character(0),
    Sensor = character(0),
    X = numeric(0),
    Y = numeric(0),
    stringsAsFactors = FALSE
  )

  for (i in seq_len(nrow(df))) {

    for (sensor in sensors) {

      x_column <- paste0(sensor, "_X")
      y_column <- paste0(sensor, "_Y")

      result <- rbind(
        result,
        data.frame(
          Time = as.character(df[[time_col]][i]),
          Sensor = sensor,
          X = as.numeric(df[[x_column]][i]),
          Y = as.numeric(df[[y_column]][i]),
          stringsAsFactors = FALSE
        )
      )
    }
  }

  expected_rows <- nrow(df) * length(sensors)

  if (nrow(result) != expected_rows) {
    stop("Reshaped data has an unexpected number of rows.")
  }

```

STDIN

Input for the program (Optional)

Output:

```
===== RESHAPED STRAIN DATA =====

   Time  Sensor    X   Y
1    T1 SensorA 10.2 5.1
2    T1 SensorB 12.4 6.0
3    T1 SensorC 14.1 7.2
4    T1 SensorD 16.3 8.0
5    T2 SensorA 10.5 5.4
6    T2 SensorB 12.8 6.2
7    T2 SensorC 14.5 7.5
8    T2 SensorD 16.7 8.3
9    T3 SensorA 10.8 5.7
10   T3 SensorB 13.1 6.5
11   T3 SensorC 14.9 7.8
12   T3 SensorD 17.0 8.6

===== DIMENSION CHECK =====

Number of rows: 12
Number of columns: 4
Expected rows: 12

===== TIME-SENSOR MAPPING CHECK =====


    SensorA SensorB SensorC SensorD
  T1       1       1       1       1
  T2       1       1       1       1
  T3       1       1       1       1
```

```r
performance_data <- data.frame(
  Sensor = c(
    "SensorA", "SensorA", "SensorA", "SensorA",
    "SensorB", "SensorB", "SensorB", "SensorB"
  ),
  Timestamp = c(
    "T1", "T1", "T2", "T2",
    "T1", "T1", "T2", "T2"
  ),
  Bitrate = c(
    "Low", "High", "Low", "High",
    "Low", "High", "Low", "High"
  ),
  Performance = c(
    80, 90, 85, 88,
    70, 76, 74, 82
  ),
  stringsAsFactors = FALSE
)

molten_data <- reshape(
  performance_data,
  varying = "Performance",
  v.names = "Value",
  timevar = "Metric",
  times = "Performance",
  direction = "long"
)

row.names(molten_data) <- NULL
molten_data$id <- NULL

cat("===== MOLTEN DATA =====\n\n")

print(molten_data)

mean_data <- aggregate(
  Value ~ Sensor + Timestamp,
```

STDIN

Input for the program (Optional)

Output:

```
===== MOLTEN DATA =====

   Sensor Timestamp Bitrate      Metric Value
1 SensorA        T1     Low Performance    80
2 SensorA        T1    High Performance    90
3 SensorA        T2     Low Performance    85
4 SensorA        T2    High Performance    88
5 SensorB        T1     Low Performance    70
6 SensorB        T1    High Performance    76
7 SensorB        T2     Low Performance    74
8 SensorB        T2    High Performance    82

===== MEAN VALUES =====

   Sensor Timestamp Mean_Performance
1 SensorA        T1             85.0
2 SensorB        T1             73.0
3 SensorA        T2             86.5
4 SensorB        T2             78.0

===== VARIANCE VALUES =====

   Sensor Timestamp Variance_Performance
1 SensorA        T1                 50.0
2 SensorB        T1                 18.0
3 SensorA        T2                  4.5
4 SensorB        T2                 32.0

===== MEAN AND VARIANCE TOGETHER =====

   Sensor Timestamp Mean_Performance Variance_Performance
```

```r
df_control <- data.frame(
  SystemID = c("S01", "S01", "S02", "S03"),
  Timestamp = as.POSIXct(
    c(
      "2026-08-12 10:00:01",
      "2026-08-12 10:00:05",
      "2026-08-12 10:00:10",
      "2026-08-12 10:00:15"
    ),
    format = "%Y-%m-%d %H:%M:%S"
  ),
  Status = c("Normal", "Error", "Normal", "Error"),
  ControlValue = c(101, 102, 98, 105),
  stringsAsFactors = FALSE
)

df_telemetry <- data.frame(
  SystemID = c("S01", "S01", "S02", "S02", "S03"),
  Timestamp = as.POSIXct(
    c(
      "2026-08-12 10:00:03",
      "2026-08-12 10:00:07",
      "2026-08-12 10:00:12",
      "2026-08-12 10:00:20",
      "2026-08-12 10:00:13"
    ),
    format = "%Y-%m-%d %H:%M:%S"
  ),
  Status = c("Normal", "Error", "Error", "Normal", "Error"),
  Temperature = c(72.1, 74.5, 73.2, 75.8, 76.4),
  stringsAsFactors = FALSE
)

tolerance_seconds <- 2

matched_rows <- list()
used_telemetry <- logical(nrow(df_telemetry))

```

STDIN

Input for the program (Optional)

Output:

```
  SystemID   Control_Timestamp Telemetry_Timestamp Control_Status
1      S01 2026-08-12 10:00:01 2026-08-12 10:00:03         Normal
2      S01 2026-08-12 10:00:05 2026-08-12 10:00:07          Error
3      S02 2026-08-12 10:00:10 2026-08-12 10:00:12         Normal
4      S02                <NA> 2026-08-12 10:00:20           <NA>
5      S03 2026-08-12 10:00:15 2026-08-12 10:00:13          Error
  Telemetry_Status ControlValue Temperature Time_Difference_Seconds
1           Normal          101        72.1                       2
2            Error          102        74.5                       2
3            Error           98        73.2                       2
4           Normal           NA        75.8                      NA
5            Error          105        76.4                       2
    Match_Status Combined_Status
1        Matched          Normal
2        Matched           Error
3        Matched        Conflict
4 Telemetry_Only          Normal
5        Matched           Error

===== MATCH SUMMARY =====


       Matched Telemetry_Only
             4              1

===== STATUS SUMMARY =====


Conflict    Error   Normal
       1        2        2
```